

Version Control and Reproducibility

14.33 Research and Communication in Economics

Jon Cohen

September 23, 2026

Agenda

1. Use the command line to describe actions precisely
2. Use Git to record useful versions of a research project
3. Use branches to separate exploratory work
4. Make one version reproducible by another person

The goal

At the end of the semester, you should be able to answer:

1. What changed?
2. Why did it change?
3. Which version produced this result?
4. Can another person produce it again?

Organization without version control

```
1 analysis.py
2 analysis_new.py
3 analysis_final.py
4 analysis_final_fixed.py
5 analysis_final_final_no_seriously.py
```

Prediction: Which file produced the table in the paper?

Version control records project states

- 1 ● Estimate baseline model
- 2 |
- 3 ● Correct treatment coding
- 4 |
- 5 ● Import raw survey data

Each **commit** records a project snapshot and an explanation.

Version control changes how you work

Without Git

Copy files to preserve versions

Recall last month's changes

Fear an unsuccessful experiment

Lose the last working version

With Git

Record a commit

Inspect the history

Create a branch

Return to a prior commit

A reproducible result

Another person can use the same inputs and instructions to produce the same result.

Git preserves the version. Git does not preserve the complete computing environment.

Command line interface

Three parts of a command-line session

Terminal

Displays text

Shell

Interprets commands

Program

Performs an action

```
1 $ git status
```

A command has a structure

git log --online analysis.py

Program	Action	Option	Argument
---------	--------	--------	----------

Location matters

```
1 $ pwd
2 /Users/you/research
3
4 $ ls
5 README.md  analysis.py  data  output
6
7 $ cd data
8 $ pwd
9 /Users/you/research/data
```

Most Git commands act on the repository that contains the current directory.

Prediction: where is the file?

```
1 $ pwd
2 /Users/you/research/code
3
4 $ ls ../data/raw
5 survey.csv
6
7 $ python clean.py ../data/raw/survey.csv
```

What path does `../data/raw/survey.csv` identify?

A small command-line vocabulary

Goal	Command
Show the current directory	<code>pwd</code>
List files	<code>ls</code>
Change directory	<code>cd <path></code>
Create a directory	<code>mkdir <path></code>
Read a text file	<code>cat <file></code>
Get help	<code><command> --help</code>

Prefer a recorded procedure

Manual workflow

Open the CSV. Delete three rows. Rename two columns. Save a new file.

Scripted workflow

```
1 python code/clean.py data/raw/survey.csv data/derived/survey.csv
```

Git's data model

Git and GitHub are different

Git

- Version-control software
- Runs on your computer
- Does not require the internet

GitHub

- Hosts Git repositories
- Supports sharing and review
- Requires a network connection

A Git repository contains a graph



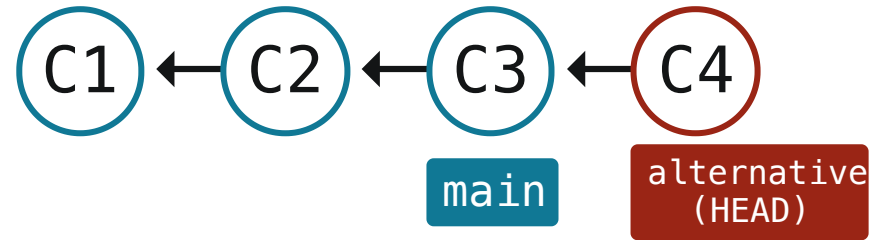
- Each commit records a snapshot
- Each commit identifies its parent
- The history records relationships, not only dates

A commit is more than a saved file

```
1 commit 8e12c4a
2 Parent: 47d906b
3 Author: Student Name
4 Date: 2026-09-16
5
6     Correct treatment coding
```

A useful commit records one coherent change.

Branches are movable labels



HEAD identifies your current branch and commit.

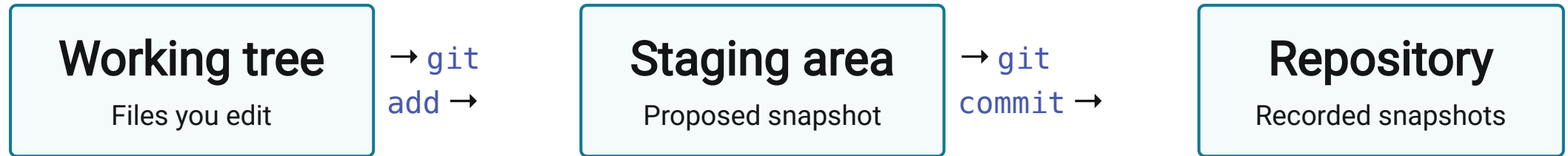
Prediction: what moves?

```
1 git switch -c alternative-specification
2 # edit analysis.py
3 git add analysis.py
4 git commit -m "Estimate quadratic specification"
```

After the commit:

- Which branch label moves?
- Does **main** move?
- Which commit does **HEAD** identify?

Three states before a commit



Prediction: one file, three versions

```
1 # analysis.py contains model A
2 git add analysis.py
3
4 # analysis.py is edited and now contains model B
5 git commit -m "Estimate baseline model"
```

Which model does the commit contain?

The core inspection loop

```
1 git status
2 git diff
3 git add analysis.py
4 git diff --staged
5 git commit -m "Estimate baseline model"
```

Inspect before you change Git's state.

An individual research workflow

Start with a project directory

```
1 wage-project/  
2 |— README.md  
3 |— .gitignore  
4 |— code/  
5 |   |— clean.py  
6 |   └— analyze.py  
7 |— data/  
8 |   |— raw/  
9 |   └— derived/  
10 └— output/
```

```
1 cd wage-project  
2 git init
```

Inspect the initial state

```
1 $ git status --short
2 ?? .gitignore
3 ?? README.md
4 ?? code/
```

?? means that Git does not track the file.

Select the first snapshot

```
1 git add README.md .gitignore code/  
2 git diff --staged  
3 git commit -m "Create analysis workflow"
```

The message completes the sentence: **This commit will...**

Read the history

```
1 $ git log --oneline --decorate
2 3cb418f (HEAD -> main) Create analysis workflow
```

```
1 git show 3cb418f
2 git log --oneline -- code/analyze.py
```

History is useful only when commits and messages are informative.

Use a branch for an uncertain direction

```
1 git switch -c alternative-specification
2
3 # edit, run, and inspect the analysis
4
5 git add code/analyze.py
6 git commit -m "Estimate quadratic specification"
```

main still identifies the prior working version.

Compare before you decide

```
1 git diff main..alternative-specification
2 git log --oneline --graph --all --decorate
```

Then choose:

- Keep developing the branch
- Merge the branch into **main**
- Leave the branch for later
- Delete the branch after rejecting the idea

Returning to working code

```
1 git log --oneline  
2 git show <commit>
```

First ask:

1. Are the changes committed?
2. Do I want to inspect, restore, or reverse them?
3. Has another person received the commits?

Prediction: what would you protect?

You have working code on `main`. You want to try a new estimator that may require many edits.

Before the experiment, what would you do?

1. Confirm that the working version is committed
2. Create a branch
3. Make and test the edits on that branch

Keep unsuitable files out of Git

```
1 # Private or restricted data
2 data/raw/
3
4 # Generated output
5 output/
6
7 # Local environments and system files
8 .venv/
9 .DS_Store
```

Commit **.gitignore** before Git tracks these files.

Remote repositories provide another copy

```
1 git clone <url>  
2 git fetch  
3 git push
```

- **clone**: create a local repository from a remote repository
- **fetch**: download remote commits and labels
- **push**: send local commits and update a remote label

When you collaborate later

- Give each change a separate branch
- Make small, coherent commits
- Fetch before you integrate work
- Review the diff before you merge
- Use a pull request when review or discussion is useful

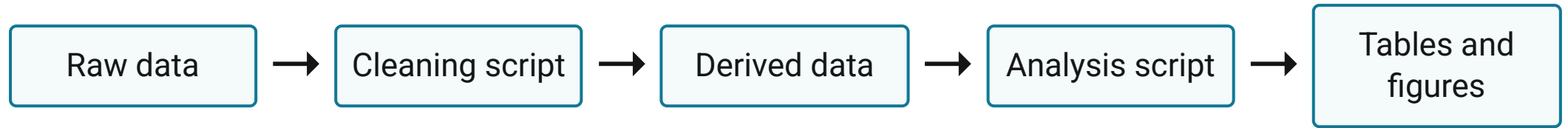
Reproducible research

A Git repository is necessary, not sufficient

1	Exact code	✓
2	Input data	?
3	Run order	?
4	Package versions	?
5	Random seed	?
6	System tools	?

The commit identifies **what you wrote**, not every condition required to run it.

Make the pipeline explicit



Each derived file should have a documented path back to its inputs and code.

Record the computing environment

Analysis Your scripts and notebooks

Packages pandas <version>, numpy <version>, ...

Language Python <version>

System Operating system and external tools

Different environments can produce different results.

Isolation and pinning solve different problems

Practice

Question it answers

Virtual environment Which packages belong to this project?

Version pinning Which package versions produced the result?

- 1 Project A → isolated environment → recorded versions
- 2 Project B → isolated environment → different recorded versions

Control randomness

```
1 import numpy as np
2
3 rng = np.random.default_rng(1433)
4 sample = rng.choice(observations, size=500, replace=True)
```

- Set and record a seed
- Use the same random-number generator
- Document intentional variation across runs

Use project-relative paths

```
1 # Fragile: specific to one computer
2 data = read_csv("/Users/name/Desktop/project/data/survey.csv")
3
4 # Portable: relative to the project
5 data = read_csv("data/raw/survey.csv")
```

Do not make another user reproduce your personal directory structure.

The README is the entry point

A new reader should learn:

1. What the project does
2. Which data it requires
3. How to create the environment
4. Which command starts the workflow
5. Where the outputs appear

Prediction: will this project reproduce?

- 1 ✓ Code is committed
- 2 ✓ README says: "Run analysis.py"
- 3 ✓ Random seed is set
- 4 ✗ Raw data location is not documented
- 5 ✗ Package versions are not recorded
- 6 ✗ clean_final.csv was edited manually

What must change before another researcher can reproduce the result?

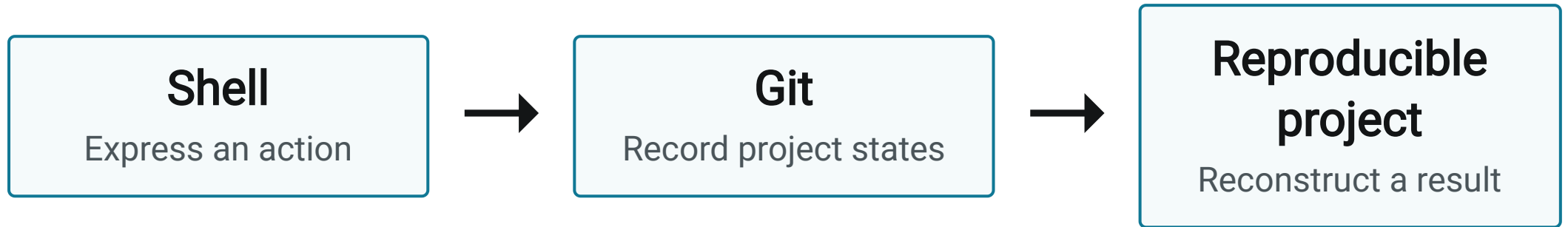
Directing an AI agent

Do not ask only: **“Fix my analysis.”**

Specify the process:

1. Inspect `README.md`, `git status`, and the recent history
2. Create a branch from the current clean commit
3. Change only the stated files
4. Run the documented checks
5. Show the diff and explain each commit

One mental model



Organization is part of the research method.

Continue learning

- The Missing Semester: the shell
- The Missing Semester: command-line environment
- The Missing Semester: version control
- Git for Ages 4 and Up
- Pro Git